

Assignment from Guillermo

1. The assignment

Dataset: <https://www.kaggle.com/datasets/dmay01/usefuldata>

The dataset is NFL Next Gen Stats tracking data (10 Hz player positions) paired with PFF scouting grades for a set of pass plays. It comes with no pre-defined target column, so the first job was deciding **what to predict and why** before deciding **how**.

2. TL;DR

Rather than picking one arbitrary target, I ran an EDA that surfaced five genuinely different, well-supported questions about pass-protection behavior, and built a dedicated model for each — all trained from the two raw source files with zero shared feature pipeline for now, so every result can be traced back to the raw rows it came from.

#	Notebook	Question	Type	Rows	Metric	Result	vs. baseline
D	<code>02_model_player_role.ipynb</code>	What role was this player performing, from movement alone?	5-class	187,726	Macro-F1	0.788	0.197 (dummy) / 0.661 (LR)
A	<code>03_model_pressure_allowed.ipynb</code>	Will this blocker get beaten?	binary	44,403	PR-AUC	0.295	0.067 (dummy) / 0.201 (LR)
B	<code>04_model_pressure_generated.ipynb</code>	Will this rusher win the matchup?	binary	36,263	PR-AUC	0.509	0.118 (dummy) / 0.390 (LR)
C	<code>05_model_block_type.ipynb</code>	Can blocking technique be read from movement alone?	11-class	45,751	Macro-F1	0.293	0.094 (dummy) / 0.218 (LR)
E	<code>06_survival_time_to_pressure.ipynb</code>	How long does a block last before it breaks?	survival	44,403	C-index	0.8028	0.500 (random)

Every model beats its logistic-regression benchmark, which in turn clears the dummy baseline by a wide margin — the tracking data reliably contains signal for each question, at a level appropriate to how hard each question actually is. These are the GroupKFold-cross-validated, hyperparameter-tuned results (see §6) from the current notebooks — D, C, and E moved up from their original fixed-hyperparameter runs; A is essentially flat (0.297 → 0.295, within the noise of a 9K-row test set at 6.6% positive rate); E’s jump (0.75 → 0.80) is mostly the CV picking a much lighter penalizer (0.01 vs. the old fixed 0.1) plus fixing the in-sample-evaluation bug described in §6.

3. Dataset

Source: [kaggle.com/datasets/dmay01/usefuldata](https://www.kaggle.com/datasets/dmay01/usefuldata) — NFL pass plays only.

data/pffScoutingData.csv	188,254 rows × 15 cols – one row per (game, play, player): a human scout's grade of what that player did on that play
data/team_information.csv	36 rows – team metadata
data/tracking_parquets/week*.parquet	~8M rows – one row per (game, play, player, 1/10s frame): raw x, y, speed, acceleration, orientation, direction at 10 Hz

The two sources describe the *same plays* from two angles: PFF has no sense of time within a play, tracking has no sense of outcome — the modeling work in this project is entirely about reconciling those two views. Every player on every play in the dataset falls into exactly one of five roles (`pff_role`): **Coverage**, **Pass Block**, **Pass Route**, **Pass Rush**, **Pass** (the QB). Full domain background, coordinate system, and column definitions are in [docs/](#).

4. Repository layout

```
.
├── README.md
├── Makefile                every workflow below as a `make <target>` – see `make help`
├── pytest.ini
├── requirements.txt        dev / notebook environment
├── requirements-serving.txt lean runtime deps for the Docker image
├── Dockerfile
├── data/                  raw CSV + parquet (not included in this submission – see §8)
├── docs/
│   ├── 00_DocCenter.md    index of the PDF copies below
│   ├── UseCases.md        the full investigative narrative behind the 5 use cases
│   ├── Dictionary.md      column-by-column definitions, by source file
│   ├── GeneralKnowledge.md football/domain primer (rules, positions, play structure)
│   ├── TrackingCoordinates.md field coordinate system, orientation vs. direction
│   ├── assets/            diagrams referenced by the docs above
│   └── pdf/               every .md in this project, rendered to PDF – `make docs-pdf`
├── scripts/
│   └── build-pdfs.sh      pandoc + headless Chrome, no LaTeX needed (used by docs-pdf)
├── notebooks/
│   ├── 01_eda.ipynb       exploratory analysis – validates all 5 use cases
│   ├── 02_model_player_role.ipynb D – player_role (multiclass)
│   ├── 03_model_pressure_allowed.ipynb A – pressure_allowed (binary)
│   ├── 04_model_pressure_generated.ipynb B – pressure_generated (binary)
│   ├── 05_model_block_type.ipynb C – block_type (multiclass)
│   └── 06_survival_time_to_pressure.ipynb E – time_to_pressure (survival)
├── artifacts/
│   ├── player_role/       model.joblib, lr_model.joblib, label_encoder.joblib,
│   ├── pressure_allowed/  feature_table.parquet, feature_importance.csv,
│   ├── pressure_generated/ metrics_report.json – one set per use case
│   ├── block_type/
│   └── time_to_pressure/  cox_model.joblib, km_overall.joblib, hazard_ratios.csv
├── serving/               FastAPI service + Streamlit demo over the saved artifacts (see §9)
│   ├── registry.py, schemas.py, main.py, streamlit_app.py, utils.py
│   └── predictors/        one module per use case
└── tests/                 pytest suite for the API (see §9)
```

Each notebook is self-contained: it reads the raw CSV/parquet directly, engineers its own features, defines its own target, trains, evaluates, and writes its artifacts to `artifacts/<use_case>/`. Nothing is imported from a shared feature-engineering module — that was a deliberate constraint, so any result in this README can be traced back to the exact raw rows and transformations that produced it.

Every `.md` file in the project has a PDF counterpart under `docs/pdf/` — `00_DocCenter.pdf` is the index. Regenerate them anytime with `make docs-pdf` (uses `pandoc` + headless Chrome at build time only; the reader of the PDF needs neither).

5. Why five models instead of one

The full reasoning is written up narratively in `docs/UseCases.md`; the short version:

1. **EDA first.** The PFF file's five roles have almost mutually-exclusive column fill patterns (a rusher's columns are ~100% filled, a blocker's are ~0% filled, and vice-versa). That raised the question: if the *labels* separate this cleanly, does the *movement* separate them too? That became model **D**.
2. **D succeeding** (Macro-F1 0.782 vs. a 0.197 dummy baseline) was the gate: if movement alone can't distinguish roles, there's no reason to trust movement-derived features for anything downstream. It passed, so I moved to pass protection specifically.
3. **A** asks the direct offensive question (will the blocker get beaten), **B** asks the mirrored defensive question (will the rusher win) — and B is *not* just A reversed, because `pff_nflIdBlockedPlayer` only names one opponent per blocker and breaks down for double-teams and unblocked rushers. B instead computes nearest-blocker distance frame-by-frame, which generalizes to any rush structure.
4. **C** asks a narrower, harder question: is blocking *technique* itself visible in movement, using blocker-only features (no rusher information at all). The literal PP finding is that technique is largely a **pre-snap** decision (play call, formation) — so a modest Macro-F1 here is itself informative, not a bug.
5. **E** exists because 93.4% of blocks never fail — the play ends before the block breaks down. A classifier would have to either discard 93% of the data or pretend “survived this play” means “invulnerable forever.” Both are wrong, so this uses a Kaplan-Meier curve plus a Cox Proportional Hazards model instead, which correctly uses both the failure-time rows and the right-censored rows.

A sixth question — a continuous 0–100 “block quality” score, scoped as regression to keep more nuance than the binary pressure label — is documented in `docs/UseCases.md` as a natural next step but was **not implemented** in this submission; see §7.

6. Modeling approach (applies across all five notebooks)

- **Baseline → Logistic Regression → HistGradientBoostingClassifier.** Every classification notebook trains all three, so the primary model's lift over both a naive baseline and an interpretable linear benchmark is always visible.
- **Group-aware split, everywhere — including cross-validation.** All splits are `GroupShuffleSplit` on `gameId`, never a row-level split. Rows from the same game share players, scheme, and context, so a row-level split would leak correlated examples across train/test and overstate performance. The same discipline applies one level down: hyperparameter search uses `GroupKFold`, not plain `KFold`, so no game ever appears in both a fold's train and validation portion.
- **Hyperparameter tuning via cross-validation, not fixed defaults.** The HGBT step in notebooks 02–05 runs a `RandomizedSearchCV` (5-fold `GroupKFold`, ~15 candidate configs over `max_depth / learning_rate / min_samples_leaf / max_leaf_nodes`) on the **training split only** — the held-out test set stays untouched for every reported metric. This is deliberately a modest search, not an exhaustive grid, given the feature engineering upstream is already the expensive part of each notebook. Notebook 06 (Cox) got a bigger fix: it previously fit on the *entire* feature table with no split at all, so its C-index was in-sample/optimistic — it now gets a proper `gameId` train/test split plus a 5-fold `GroupKFold` search over the `penalizer` term, with the final C-index reported on the held-out test split.
- **Imbalance handled explicitly, not hidden.** Positive rates are 6.6–11.7% for the binary use cases; `class_weight="balanced"` is used throughout, and the reported metric is **PR-AUC** (binary) or **Macro-F1** (multiclass) rather than accuracy, since accuracy is close to meaningless on data this skewed.
- **Leakage removed by construction, per problem.** e.g. in A, `pff_hitAllowed / pff_hurryAllowed / pff_sackAllowed` directly define the label and are dropped from features; `separation_at_end` is dropped everywhere because it's measured after the outcome is already resolved; in E, `window_duration_s` is the duration itself and is excluded as a covariate to avoid circularity.
- **Feature importance via SHAP, not `.feature_importances_`.** `HistGradientBoostingClassifier` doesn't expose that attribute, so every notebook uses `shap.TreeExplainer` instead (mean ISHAP value per feature — averaged over classes for the multiclass use cases).
- **Every artifact is reproducible from raw data alone**, and each notebook saves its trained model(s), encoders, feature table, feature-importance ranking, and a `metrics_report.json` to `artifacts/<use_case>/`.

7. Results in detail

D — `player_role` (multiclass, foundation model)

Self-movement-only features (speed, acceleration, orientation/direction variability, displacement) — no PFF outcome fields used, since the target *is* a PFF field (`pff_role`) and using derived PFF columns would be circular.

Metric	Dummy	Logistic Regression	HGBT (primary)
Macro-F1	0.197	0.661	0.788
CV Macro-F1 (5-fold, train only)	—	—	0.777 ± 0.006

Tuned params: `max_depth=6`, `learning_rate=0.047`, `max_leaf_nodes=43`, `min_samples_leaf=27` . Top SHAP feature: `o_dir_delta_mean` (gap between body orientation and travel direction — the classic backpedaling-defender signal). Per-class F1 ranges 0.63 (Pass/QB, smallest class) to 0.84 (Pass Block).

A — `pressure_allowed` (binary, offensive side)

One row per blocker↔rusher assignment (44,403 rows). Features combine both players' movement plus separation-at-snap, closest-approach distance, and closing velocity.

Metric	Dummy	LR	HGBT (primary)
PR-AUC	0.067	0.201	0.295
CV PR-AUC (5-fold, train only)	—	—	0.321 ± 0.013
ROC-AUC	—	—	0.850

Tuned params: `max_depth=6`, `learning_rate=0.028`, `max_leaf_nodes=50`, `min_samples_leaf=49` . Top SHAP features: `rusher_speed_mean`, `closing_velocity`, `separation_min` — consistent with the EDA finding that pressure-allowed blocks show a closer average approach (0.655 yd) than clean blocks (0.829 yd). Test PR-AUC sits below the CV mean here (0.295 vs. 0.321) — expected: CV is measured on train-only folds, so a single held-out test set of 9K rows at 6.6% positive will naturally show more variance than the 5-fold average.

B — `pressure_generated` (binary, defensive side — not A reversed)

One row per rusher per play (36,263 rows). Nearest-blocker distance is computed frame-by-frame instead of relying on the single named-opponent field, so it generalizes across single blocks, double-teams, and unblocked rushers.

Metric	Dummy	LR	HGBT (primary)
PR-AUC	0.118	0.390	0.509
CV PR-AUC (5-fold, train only)	—	—	0.468 ± 0.019
ROC-AUC	—	—	0.873

Tuned params: `max_depth=6`, `learning_rate=0.047`, `max_leaf_nodes=43`, `min_samples_leaf=27` . Top SHAP features: `rusher_displacement`, `nearest_closing_vel`, `rusher_speed_mean` .

C — `block_type` (multiclass, blocker-only features)

Rare techniques (< 150 occurrences) collapsed into `OTHER` , giving 11 classes. **Deliberately excludes all rusher information** — the point is to isolate what the blocker's own motion reveals about technique.

Metric	Dummy	LR	HGBT (primary)
Macro-F1	0.094	0.218	0.293
CV Macro-F1 (5-fold, train only)	—	—	0.285 ± 0.011

Tuned params: `max_depth=6`, `learning_rate=0.047`, `max_leaf_nodes=43`, `min_samples_leaf=27`. This is the intended finding, not underperformance — now landing right at the ≈ 0.31 target the pre-modeling EDA predicted (see `docs/UseCases.md`): literature and the result agree that blocking technique is decided mostly **pre-snap** (play call, formation), so post-snap motion should not be expected to fully recover it. The practical reframe is a **deviation detector** — flag movement that looks unusual for the technique that was called — rather than a technique classifier.

E — `time_to_pressure` (survival analysis)

Same population as A (44,403 assignments), but instead of a binary outcome this models **how long** a block holds. 93.4% of rows are right-censored (block never failed before the play ended).

Metric	Value
C-index (Cox PH, held-out test)	0.8028
CV C-index (5-fold, train only)	0.803 $\pm$ 0.013
Tuned penalizer	0.01
Median survival time	11.3 s after snap
Events / censored	2,949 / 41,454

The CV search picked a much lighter penalizer (0.01) than the original fixed 0.1, which — combined with fixing the in-sample-evaluation bug described in §6 — moved the C-index from 0.75 to a genuinely held-out 0.80. Top hazard-ratio findings now (all HR relative to the Cox baseline, $p < 0.05$ except where noted): `bt_SW` (HR ≈ 2.62) and `rusher_speed_mean` (HR ≈ 2.52) raise the moment-to-moment failure hazard the most, while `separation_min` is the strongest protective factor (HR ≈ 0.42) — a closer minimum approach shortens survival substantially, consistent with A and B's binary results, now expressed as a time-varying risk instead of a single yes/no.

8. Reproducing this

```
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

Place the three raw files from the Kaggle dataset under `data/` (`pffScoutingData.csv`, `team_information.csv`, `tracking_parquets/week1.parquet` ... `week8.parquet` — not included in this submission due to size). Then run the notebooks in order:

```
01_eda.ipynb    → validates the five use cases against the raw data
02 → 06         → each one independently: raw data → features → model → artifacts/<use_case>/
```

Each notebook can be run on its own; only `04` reads a saved metric from `03`'s artifacts (for a side-by-side A-vs-B comparison cell), so `03` should run first if that comparison is wanted.

9. Serving

The five trained models are served behind a FastAPI app, with a Streamlit demo on top of the same prediction code (no duplicated logic between the two).

Run locally:

```
uvicorn serving.main:app --reload
# Swagger UI at http://127.0.0.1:8000/docs
```

Run with Docker (build copies `requirements-serving.txt` + `serving/` + `artifacts/` — notebooks/docs/data/tests are excluded via `.dockerignore` to keep the image lean):

```
docker build -t pass-protection-api .
docker run -p 8000:8000 pass-protection-api
```

Run the demo dashboard:

```
streamlit run serving/streamlit_app.py
```

Run the test suite (`pytest` / `httpx` are in `requirements.txt`):

```
pytest tests/ -v
```

Each `/predict/{use-case}` endpoint accepts exactly the feature set its notebook trained on (see `serving/schemas.py`) and applies the same decision logic — including the threshold tuned on the held-out test set for the two binary classifiers, not a default 0.5. An unseen categorical value (e.g. an unrecognized `pff_blockType`) returns a 422 listing the valid values instead of a raw 500 — the one real “breaks in prod” edge case in these artifacts, and it’s covered by `tests/test_validation.py`. `serving/predictors/time_to_pressure.py` is the one non-trivial predictor: it reconstructs the Cox model’s block-type dummy encoding at inference time by reindexing to `cph.params_.index`, so it stays correct without hardcoding which dummy columns exist.

This was built and verified end-to-end this session: `pytest tests/` (14/14 passing) against the real trained artifacts, a live `uvicorn` smoke test, a full `docker build` + `docker run` round-trip hitting the containerized API from outside, and a headless Streamlit boot check.

10. Limitations and next steps

- **The 0–100 “block quality” regression** scoped in `docs/UseCases.md` (a continuous alternative to the binary pressure label) was designed but not implemented in this submission — the natural next model to add.
- **B’s nearest-blocker heuristic** is a reasonable proxy for double-teams and free rushers but doesn’t model blocking assignment logic explicitly; a natural extension is comparing it against a learned assignment model.
- Full per-column definitions, the coordinate system, and domain background for anyone unfamiliar with American football are in `docs/`